

Software Design – Klassediagram

Dries Van den Brande, Stef De Bie, Lars Verheyden

1 Klassediagram

1.1 Basisconcepten

Klasse

Een verzameling van objecten met overeenkomstige eigenschappen.

Kennen:

- Een klassebeschrijving bevat de naam van de klasse en haar eigenschappen.
- Klassen worden door de modelleur bepaald, bijvoorbeeld via classificatie.

Object / instantie

Een object heeft een zelfstandig bestaan, in werkelijkheid of conceptueel.

Kennen:

- **Identiteit:** elk object is uniek.
- **Toestand:** de data van het object.
- **Gedrag:** de operaties van het object.
- **Encapsulation:** interne details worden verborgen.
- **Abstraction:** het object wordt via zijn interface benaderd.
- **Instantie** is een andere naam voor object, vooral wanneer men wil aangeven tot welke klasse het object behoort.

Attribuut

Informatie die door een object beheerd wordt.

Kennen:

- De waarden van alle attributen samen vormen de toestand van een object.
- Twee objecten kunnen dezelfde toestand hebben en toch verschillende objecten zijn, omdat elk object een eigen identiteit heeft.
- Een attribuut heeft een bepaald type.
- Voorbeelden bij een persoon: naam, geboortedatum en gewicht.
- In UML kan een attribuut een multipliciteit hebben.

Operatie

Een operatie beschrijft een service die door een object geleverd wordt.

Kennen:

- Een operatie moet behoren tot de verantwoordelijkheden van het object.

- De verzameling operaties vormt het gedrag van het object.
- Een operatie kan argumenten hebben.
- Een operatie kan een resultaat opleveren.
- Objecten communiceren door boodschappen te sturen tussen zender en ontvanger.

Associatie

Een structurele relatie tussen twee klassen.

Kennen:

- Een instantie van de ene klasse is gedurende een groot deel van haar bestaan verbonden met een instantie van de andere klasse.
- Tussen twee klassen kunnen meerdere associaties bestaan.
- De naam van de associatie staat langs de associatielijn.
- De leesrichting kan aangeduid worden.
- Een **link** is een instantie van een associatie.

Rol

De rol beschrijft een associatie-einde.

Kennen:

- Een object kan in verschillende associaties verschillende rollen spelen.
- Wanneer de rolnaam wordt weggelaten, geldt de klassenaam als rolnaam.

Multipliciteit

Geeft aan hoeveel instanties aan een associatie-einde of attribuut kunnen voorkomen.

Kennen:

- Multipliciteit kan met symbolen of expliciete cijfers worden weergegeven.
- Multipliciteit kan aan beide uiteinden van een associatie staan.
- Bij een attribuut wordt de multipliciteit tussen vierkante haken geplaatst.

Generalisatie

Het classificeren van klassen volgens een **IS-A**-relatie.

Kennen:

- Zoek klassen met gemeenschappelijke kenmerken.
- **Superklasse** = generalisatie = basisklasse.
- **Subklasse** = specialisatie = afgeleide klasse.

Overerving

Iedere subklasse erft de eigenschappen van haar superklasse.

Kennen:

- Attributen en operaties van de superklasse worden geërfd.
- Overerving volgt de IS-A-relatie.

Liskov's substitutieprincipe

Op iedere plaats waar een instantie van een klasse gebruikt kan worden, kan een instantie van een subklasse gebruikt worden.

Kennen:

- Het principe hoort bij de IS-A-relatie.

Commentaar

Extra uitleg in een UML-diagram.

Kennen:

- Commentaar kan in een notebbox geplaatst worden.

Klassediagram

Een diagram dat de klassen in een systeem toont.

Kennen:

- Bevat klassen.
- Kan attributen bevatten.
- Kan operaties bevatten.
- Kan associaties bevatten.

Objectdiagram

Een afbeelding van objecten op een bepaald tijdstip.

Kennen:

- De objecten zijn gecreëerd volgens de structuur van het klassediagram.
- Een objectdiagram toont instanties van het klassediagram.

1.2 Geavanceerde concepten

Afgeleid attribuut

Een attribuut waarvan de waarde afgeleid wordt uit andere informatie.

Kennen:

- De waarde kan afgeleid worden uit andere attributen, associaties of objecten.
- De waarde hoeft niet noodzakelijk opgeslagen te worden.
- Het is informatie waar een object om gevraagd kan worden.
- Voorbeeld: /leeftijd.

Zichtbaarheid

Geeft aan in hoeverre een attribuut of operatie door de omgeving van een object gekend is.

Kennen:

- - = private
- + = public
- # = protected

- $\sim$ = package
- Inkapseling probeert zoveel mogelijk interne informatie te verbergen.

Bag

Een meervoudige verzameling waarbij volgorde niet belangrijk is en dubbele waarden toegelaten zijn.

Sequence

Een meervoudige verzameling met een vaste volgorde waarin dubbele waarden toegelaten zijn.

Set

Een meervoudige verzameling zonder vaste volgorde waarin iedere waarde uniek is.

OrderedSet

Een meervoudige verzameling met een vaste volgorde waarin iedere waarde uniek is.

Klasseattribuut

Een attribuut dat een eigenschap van de klasse beschrijft en niet van iedere individuele instantie.

Kennen:

- Heeft slechts één waarde, onafhankelijk van het aantal instanties van de klasse.

Klasseoperatie

Een service die door de klasse zelf geleverd wordt en niet door een individuele instantie.

Kennen:

- Een constructor is een voorbeeld dat in de slides bij klasseoperaties wordt vermeld.

Afgeleide associatie

Een associatie die uit andere informatie kan worden afgeleid.

Kennen:

- Is redundant.
- Moet spaarzaam gebruikt worden.

Richting van een associatie

Geeft aan welke klasse de andere klasse kent.

Kennen:

- Een gewone associatie impliceert in principe dat de objecten aan beide uiteinden elkaar kennen.
- Een richting kan aangeduid worden om deze navigatie te beperken.
- Dit kan consistentieproblemen helpen vermijden.

Associatieklasse

Een klasse die gekoppeld is aan een associatie.

Kennen:

- De link tussen twee instanties is een instantie van de associatieklasse.
- Wordt gebruikt wanneer de associatie zelf attributen heeft.
- Wordt gebruikt wanneer de associatie zelf operaties heeft.
- Wordt gebruikt wanneer de associatie associaties met andere klassen heeft.

Compositie

Een speciaal soort associatie waarbij één of meer klassen onderdeel zijn van een andere klasse.

Kennen:

- Een onderdeel mag in een compositierelatie slechts tot één geheel behoren.
- De levensduur van het deelobject is kleiner dan of gelijk aan de levensduur van het gehele object.
- Er is een **lifetime dependency**.

Abstracte klasse

Een klasse waarvan geen instanties kunnen bestaan.

Kennen:

- Is superklasse van andere klassen.
- Dient om een gezamenlijke interface te definiëren.
- Concrete subklassen kunnen wel instanties hebben.

Abstracte operatie

Een operatie die in een abstracte klasse gedeclareerd wordt maar daar niet volledig gedefinieerd hoeft te zijn.

Meervoudige overerving

Een situatie waarbij een klasse meer dan één superklasse heeft.

Herhaalde overerving

Een vorm van overerving die kan ontstaan binnen een hiërarchie met meerdere overervingspaden.

Kennen:

- De slides vermelden dat meervoudige en herhaalde overerving niet in moderne programmeertalen gebruikt worden.

Enumeratie

Een type waarvoor een vast aantal mogelijke waarden gedefinieerd is.

Package

Een groepering van modelementen.

Kennen:

- Is een generiek mechanisme om een softwaresysteem op te delen.
- Ieder modelement behoort tot één package.
- Associaties kunnen voorkomen tussen klassen uit verschillende packages.
- Een package is een verzameling klassen die samenwerken om aan een bepaalde verantwoordelijkheid te voldoen.
- Packages kunnen genest zijn.
- Een klasse uit een andere package kan met de notatie `PACKAGE::Klasse` aangeduid worden.
- Sinds UML 2 bestaat ook *package merge*.

Geparametriseerde klasse / template

Een klasse waarvan bepaalde zaken afhankelijk zijn van één of meer parameters.

Kennen:

- Door waarden aan de parameters te geven ontstaat een klasse waarvan instanties gemaakt kunnen worden.
- Een template beschrijft een familie van klassen.
- Wanneer een parameter een type aanduidt, kan aangegeven worden aan welk ander type dat type moet voldoen.

Constraint

Een beperking op één of meer elementen in een klassediagram.

Kennen:

- Constraints kunnen met OCL uitgedrukt worden.
- Voorbeelden uit de slides zijn leeftijdsvoorwaarden en XOR-constraints.

OCL

Object Constraint Language.

Kennen:

- Wordt gebruikt om bedrijfsregels en beperkingen bij een model uit te drukken.
- Een invariant kan in de vorm `context ... inv: ...` geschreven worden.

1.3 Werkwijze voor een klassediagram

Stappenplan

1. Identificeer alle mogelijke kandidaatklassen.
2. Selecteer de klassen uit de lijst van kandidaten.
3. Maak eventueel een modeldictionary.
4. Identificeer associaties.

5. Identificeer attributen.
6. Identificeer operaties.
7. Generaliseer met behulp van overerving.
8. Voeg eventueel OCL-constraints toe.
9. Groepeer eventueel in packages.
10. Itereer over de gedane stappen.

Kandidaatklasse identificeren

Zoek zonder beperkingen zoveel mogelijk mogelijke klassen.

Kennen:

- Methode 1: onderstreep zelfstandige naamwoorden in de tekst.
- Methode 2: organiseer een brainstormsessie.
- Betrek toekomstige gebruikers, domeinexperts en ontwikkelaars.
- Tijdens deze stap: geen kritiek.
- Niet iedere term wordt uiteindelijk een klasse; termen kunnen wel in de modeldictionary terechtkomen.

Klassen selecteren

Bepaal welke kandidaatklassen effectief in het model opgenomen worden.

Vragen:

- Heeft de kandidaat een zelfstandige rol in het probleemdomein?
- Willen we iets van deze kandidaat weten?
- Moet de kandidaat iets doen?
- Heeft de kandidaat een duidelijke verantwoordelijkheid in het systeem?

Mogelijke redenen om geen klasse te maken:

- Redundant.
- Irrelevant voor het probleemdomein.
- Te vaag.
- Niet kwantificeerbaar.
- Is eigenlijk een attribuut.
- Is eigenlijk een operatie.
- Is eigenlijk een rol.

Modeldictionary

Een verzameling waarin de termen van het model worden opgenomen.

Kennen:

- Kan klassen, attributen, operaties en andere termen bevatten.

Associaties identificeren

Zoek relaties tussen paren van klassen.

Kennen:

- Zoek kandidaatassociaties.
- Elimineer tijdelijke acties of relaties die eerder dependencies zijn.
- Controleer of afgeleide associaties echt nodig zijn.
- Koppel terug naar gebruikers en domeinexperts.
- Voeg rollen en multipliciteiten toe.
- Verwijder irrelevante associaties.

Attributen identificeren

Breid de klassen uit met relevante informatie die door hun objecten beheerd wordt.

Vragen:

- Wat zijn de dingen waar een object van weet?
- Welke informatie kunnen we aan het object vragen?

Opletten voor:

- Een attribuut kan geen objectreferentie vervangen; daarvoor gebruik je een associatie.
- Identifiers zijn vaak kunstmatige attributen.
- Neem alleen interne waarden op die voor de klant relevant zijn.
- Vermijd te veel detail.
- Denk ook aan klasseattributen.
- Voeg multipliciteit toe.
- Typen worden volgens de slides later in het ontwerp toegevoegd.

Operaties identificeren

Bepaal welk gedrag bij de verantwoordelijkheid van een klasse hoort.

Vragen:

- Wat wil ik dat het object voor mij doet?
- Wat is de verantwoordelijkheid van het object?

Generaliseren met overerving

Bekijk of bestaande klassen gemeenschappelijke eigenschappen hebben die in een nieuwe superklasse geplaatst kunnen worden.

Kennen:

- Zoek identieke attributen, operaties en associaties.
- Controleer of ze ook werkelijk dezelfde betekenis hebben.
- Generaliseren is een vorm van abstraheren.
- De slides waarschuwen dat te veel generalisatie de flexibiliteit kan verminderen.

Bedrijfsregels toevoegen

Druk business rules en beperkingen uit met constraints, bijvoorbeeld in OCL.

Kennen:

- Formuleer vragen binnen het probleemdomein.
- Wanneer belangrijke vragen niet met het model beantwoord kunnen worden, kan het model te beperkt zijn.

Klassen groeperen in packages

Deel een groot klassediagram logisch of praktisch op in packages.

Kennen:

- Binnen UML is de gekozen verdeling vrij.

Itereren

Het maken van een klassediagram is geen strikt sequentieel proces.

Kennen:

- Controleer op inconsistenties en onvolledigheid.
- Een verantwoordelijkheid of operatie zonder geschikte klasse kan wijzen op een ontbrekend modelement.
- Een constraint die niet beschreven kan worden met het bestaande model kan wijzen op een ontbrekende associatie, klasse, attribuut of operatie.
- Pas het diagram aan en doorloop eerdere stappen opnieuw.